

# FinTrack

REST API Contract — v1.0

|          |                                                                                  |
|----------|----------------------------------------------------------------------------------|
| Project  | FinTrack — personal finance tracker                                              |
| Document | API Contract v1.0 (companion to Business Requirements v1.0)                      |
| Base URL | <code>https://api.fintrack.example/api/v1</code> (placeholder)                   |
| Format   | JSON over HTTPS. All request/response bodies are <code>application/json</code> . |
| Audience | Backend (implements this) & Frontend (consumes this)                             |

**Purpose.** This is the binding contract between the backend and the frontend. Both sides implement against it independently. If either side needs a change, the contract is updated first, then the code. Field names, types, status codes, and error shapes here are authoritative.

## 1. Conventions

- **Base path:** every endpoint below is relative to `/api/v1`.
- **IDs** are strings (UUID or similar). Never assume sequential integers on the frontend.
- **Timestamps** are ISO 8601 UTC strings, e.g. `"2026-07-05T14:30:00Z"`.
- **Dates** (transaction date, budget month) without time use `YYYY-MM-DD`.
- **Money:** amounts are integers in **minor units** (cents). `1250` means 12.50. This avoids floating-point drift. Formatting to "12.50" is a frontend concern.
- **Content type:** requests with a body must send `Content-Type: application/json`.
- **Auth header:** protected endpoints require `Authorization: Bearer <accessToken>`.

## 2. Authentication Model

Two-token scheme, implemented "the real way":

- **Access token** — short-lived JWT (~15 min). Returned in the response *body*. Frontend keeps it in memory (not `localStorage`) and attaches it as a Bearer header.
- **Refresh token** — long-lived (~7 days). Sent by the backend as an `httpOnly; Secure; SameSite` cookie. The frontend never reads it; the browser sends it automatically to the refresh endpoint.

**Backend must configure CORS for cookies:** `Access-Control-Allow-Credentials: true` and an explicit origin (not `*`). The frontend must send requests with `withCredentials: true` (Axios) so the refresh cookie travels. Cookie attributes: `httpOnly`, `Secure`, and `SameSite=None` if frontend and backend are on different domains (else `Lax`).

### Session flow

- **Login/Register** → body returns `accessToken` + user; response sets refresh cookie.
- **Normal request** → frontend sends `Authorization: Bearer <access>`.
- **Access expired** → backend returns `401`; frontend calls `POST /auth/refresh` (cookie auto-sent) → gets a fresh access token → retries the original request.
- **Refresh invalid/expired** → `401` → frontend clears session, redirects to login.
- **Page reload** → access token in memory is lost → on app start, frontend silently calls `POST /auth/refresh` to restore the session.

## 3. Standard Error Format

Every error response (4xx/5xx) uses this shape:

```
// HTTP 400 / 401 / 404 / 409 / 422 ...
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Amount must be a positive number.",
    "details": [
      { "field": "amount", "message": "must be greater than 0" }
    ]
  }
}
```

`details` is present only for field-level validation errors ( `422` ); otherwise omitted.

| HTTP | error.code       | When                                                                            |
|------|------------------|---------------------------------------------------------------------------------|
| 400  | BAD_REQUEST      | Malformed JSON or missing required body.                                        |
| 401  | UNAUTHENTICATED  | Missing/invalid/expired access token.                                           |
| 403  | FORBIDDEN        | Authenticated but resource belongs to another user.                             |
| 404  | NOT_FOUND        | Resource does not exist (or not owned — see note).                              |
| 409  | CONFLICT         | E.g. email already registered; deleting an account that still has transactions. |
| 422  | VALIDATION_ERROR | Field validation failed; details populated.                                     |

Privacy note: for resources owned by another user, returning 404 (rather than 403 ) is acceptable and often preferred, so existence isn't leaked. Pick one policy and keep it consistent.

## 4. Auth Endpoints

|                                                                                                                                                                                                                                   |                   |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------|
| <div>POST</div> <div>/auth/register</div>                                                                                                                                                                                         | <div>PUBLIC</div> |
| <div>REQUEST</div> <div><pre>{   "email": "jane@example.com",   "password": "at-least-8-chars",   "displayName": "Jane" }</pre></div>                                                                                             |                   |
| <div>RESPONSE 201</div> <div><pre>{   "accessToken": "eyJhbGciOi...",   "user": { "id": "usr_1", "email": "jane@example.com", "displayName": "Jane" } } // + Set-Cookie: refreshToken=...; HttpOnly; Secure; SameSite</pre></div> |                   |
| <div>ERRORS</div> <div>409 CONFLICT email taken · 422 invalid email / weak password. On success the backend also seeds default categories (US-C2).</div>                                                                          |                   |
| <div>POST</div> <div>/auth/login</div>                                                                                                                                                                                            | <div>PUBLIC</div> |
| <div>REQUEST</div> <div><pre>{ "email": "jane@example.com", "password": "..."} </pre></div>                                                                                                                                       |                   |
| <div>RESPONSE 200</div> <div><pre>{ "accessToken": "eyJ...", "user": { "id": "usr_1", "email": "...", "displayName": "Jane" } } // + Set-Cookie: refreshToken=...</pre></div>                                                     |                   |
| <div>ERRORS</div> <div>401 UNAUTHENTICATED for wrong email or password — generic message, never reveal which.</div>                                                                                                               |                   |
| <div>POST</div> <div>/auth/refresh</div>                                                                                                                                                                                          | <div>COOKIE</div> |
| <div>No body. Reads the refresh token from the httpOnly cookie.</div>                                                                                                                                                             |                   |
| <div>RESPONSE 200</div> <div><pre>{ "accessToken": "eyJ...new..." }</pre></div>                                                                                                                                                   |                   |
| <div>ERRORS</div> <div>401 if the refresh cookie is missing/expired/invalid → frontend logs the user out.</div>                                                                                                                   |                   |
| <div>POST</div> <div>/auth/logout</div>                                                                                                                                                                                           | <div>AUTH</div>   |
| <div>Invalidates the refresh token and clears the cookie.</div>                                                                                                                                                                   |                   |
| <div>RESPONSE 204</div> <div><pre>// empty body + Set-Cookie clearing refreshToken</pre></div>                                                                                                                                    |                   |
| <div>GET</div> <div>/auth/me</div>                                                                                                                                                                                                | <div>AUTH</div>   |
| <div>Returns the current authenticated user. Useful after a silent refresh on app start.</div>                                                                                                                                    |                   |
| <div>RESPONSE 200</div> <div><pre>{ "id": "usr_1", "email": "jane@example.com", "displayName": "Jane" }</pre></div>                                                                                                               |                   |

## 5. Accounts

All endpoints require auth. A user only ever sees their own accounts.

### Account object

```
{
  "id": "acc 1",
  "name": "Main card",
  "type": "card",           // "cash" | "card" | "savings" | "other"
  "startingBalance": 100000, // minor units (= 1000.00)
  "currentBalance": 87250,   // computed: starting +/- transactions
  "createdAt": "2026-07-01T10:00:00Z"
}
```

**GET** /accounts AUTH

RESPONSE 200

{ "data": [ /\* Account[] \*/ ] }

**POST** /accounts AUTH

REQUEST

{ "name": "Main card", "type": "card", "startingBalance": 100000 }

RESPONSE 201

{ /\* Account \*/ }

ERRORS

422 name empty / invalid type / non-integer balance.

**GET** /accounts/:id AUTH

RESPONSE 200

{ /\* Account \*/ }

404 if not found / not owned.

**PATCH** /accounts/:id AUTH

REQUEST (ANY SUBSET)

{ "name": "Savings", "type": "savings", "startingBalance": 50000 }

RESPONSE 200

{ /\* updated Account \*/ }

**DELETE** /accounts/:id AUTH

RESPONSE 204

// empty

ERRORS

409 CONFLICT if the account still has transactions (US-B1). Frontend must warn the user.

## 6. Categories

### Category object

```
{
  "id": "cat_1",
  "name": "Groceries",
  "type": "expense",           // "income" | "expense"
  "color": "#2f6df6",
  "createdAt": "2026-07-01T10:00:00Z"
}
```

GET /categories AUTH

Optional query: `?type=income` or `?type=expense` to filter.

RESPONSE 200

```
{ "data": [ /* Category[] */ ] }
```

POST /categories AUTH

REQUEST

```
{ "name": "Groceries", "type": "expense", "color": "#2f6df6" }
```

RESPONSE 201

```
{ /* Category */ }
```

ERRORS

422 name empty / invalid type / invalid color.

PATCH /categories/:id AUTH

REQUEST (ANY SUBSET)

```
{ "name": "Food", "color": "#e0a020" }
```

Note: `type` should be immutable once transactions use it — reject changing type with 409 if in use.

RESPONSE 200

```
{ /* updated Category */ }
```

DELETE /categories/:id AUTH

RESPONSE 204

```
// empty
```

ERRORS

409 CONFLICT if the category is used by transactions (US-C1). Frontend warns / offers reassign.

## 7. Transactions core

### Transaction object

```
{
  "id": "txn_1",
  "type": "expense",           // "income" | "expense"
  "amount": 4550,             // minor units, always positive (= 45.50)
  "date": "2026-07-04",
  "note": "Weekly shop",     // nullable
  "category": { "id": "cat_1", "name": "Groceries", "color": "#2f6df6" },
  "account": { "id": "acc_1", "name": "Main card" },
  "createdAt": "2026-07-04T18:20:00Z"
}
```

The list embeds category/account summaries (nested) so the frontend renders rows without extra lookups. Write operations accept just the IDs.

GET /transactions AUTH

QUERY PARAMETERS (ALL OPTIONAL, COMBINABLE)

| Param      | Example    | Meaning                               |
|------------|------------|---------------------------------------|
| page       | 1          | Page number, default 1.               |
| limit      | 20         | Items per page, default 20, max 100.  |
| type       | expense    | Filter by income/expense.             |
| categoryId | cat_1      | Filter by category.                   |
| accountId  | acc_1      | Filter by account.                    |
| dateFrom   | 2026-07-01 | Inclusive start date.                 |
| dateTo     | 2026-07-31 | Inclusive end date.                   |
| sort       | -date      | Sort field; - = desc. Default -date . |

RESPONSE 200

```
{
  "data": [ /* Transaction[] */ ],
  "pagination": {
    "page": 1, "limit": 20, "total": 137, "totalPages": 7
  },
  "summary": {
    // totals for the CURRENT filter set (US-D4)
    "income": 320000, "expense": 145500, "net": 174500
  }
}
```

POST /transactions AUTH

REQUEST

```
{
  "type": "expense",
  "amount": 4550,
  "date": "2026-07-04",
  "categoryId": "cat_1",
  "accountId": "acc_1",
  "note": "Weekly shop" // optional
}
```

RESPONSE 201

```
{ /* Transaction (nested) */ }
```

ERRORS

- 422 amount ≤ 0, missing fields, invalid date.
- 422 **category type mismatch** — e.g. expense txn with an income category (core business rule).
- 404 categoryId / accountId not found or not owned.

GET /transactions/:id AUTH

RESPONSE 200

```
{ /* Transaction */ }
```

**PATCH** /transactions/:id

**AUTH**

**REQUEST (ANY SUBSET OF THE POST BODY)**

```
{ "amount": 5000, "note": "Updated" }
```

**RESPONSE 200**

```
{ /* updated Transaction */ }
```

*Same validation rules as POST apply to any provided field.*

**DELETE** /transactions/:id

**AUTH**

**RESPONSE 204**

```
// empty
```

8. Analytics

core

Dedicated endpoints so the frontend doesn't compute aggregates client-side. All require auth and accept a period.

GET

/analytics/summary

AUTH

Powers the dashboard cards (US-E1). Query: `?dateFrom=&dateTo=` (defaults to current month).

RESPONSE 200

```
{
  "totalBalance": 210500,    // across all accounts, all-time
  "periodIncome": 320000,
  "periodExpense": 145500,
  "periodNet": 174500
}
```

GET

/analytics/by-category

AUTH

Powers the pie/donut chart (US-E2). Query: `?dateFrom=&dateTo=&type=expense` .

RESPONSE 200

```
{
  "data": [
    { "categoryId": "cat_1", "name": "Groceries", "color": "#2f6df6", "total": 65000 },
    { "categoryId": "cat_2", "name": "Rent",      "color": "#b32020", "total": 80000 }
  ]
}
```

GET

/analytics/over-time

AUTH

Powers the income-vs-expense trend chart (US-E3). Query: `?months=6` (default 6).

RESPONSE 200

```
{
  "data": [
    { "month": "2026-06", "income": 300000, "expense": 120000 },
    { "month": "2026-07", "income": 320000, "expense": 145500 }
  ]
}
```



9. Budgets

OPTIONAL — stretch (Epic F)

This section is **optional**. Implement only after sections 4-8 are complete. The frontend should tolerate its absence — treat these endpoints as a later add-on.

Budget object

```
{
  "id": "bud_1",
  "categoryId": "cat_1",
  "month": "2026-07",           // YYYY-MM
  "limit": 80000,               // minor units
  "spent": 65000                // computed for the month
}
```

**GET** /budgets

AUTH

Query: ?month=2026-07 .

RESPONSE 200

```
{ "data": [ /* Budget[] with spent */ ] }
```

**POST** /budgets

AUTH

REQUEST

```
{ "categoryId": "cat_1", "month": "2026-07", "limit": 80000 }
```

RESPONSE 201

```
{ /* Budget */ }
```

ERRORS

422 category must be an expense category · 409 budget already exists for that category+month.

**PATCH** /budgets/:id

AUTH

REQUEST

```
{ "limit": 90000 }
```

RESPONSE 200

```
{ /* updated Budget */ }
```

**DELETE** /budgets/:id

AUTH

RESPONSE 204

```
// empty
```

## 10. Endpoint Summary

| Method | Path                   | Auth   | Purpose                        |
|--------|------------------------|--------|--------------------------------|
| POST   | /auth/register         | Public | Create account, auto-login     |
| POST   | /auth/login            | Public | Log in                         |
| POST   | /auth/refresh          | Cookie | New access token               |
| POST   | /auth/logout           | Auth   | End session                    |
| GET    | /auth/me               | Auth   | Current user                   |
| GET    | /accounts              | Auth   | List accounts                  |
| POST   | /accounts              | Auth   | Create account                 |
| GET    | /accounts/:id          | Auth   | Get account                    |
| PATCH  | /accounts/:id          | Auth   | Update account                 |
| DELETE | /accounts/:id          | Auth   | Delete account                 |
| GET    | /categories            | Auth   | List categories                |
| POST   | /categories            | Auth   | Create category                |
| PATCH  | /categories/:id        | Auth   | Update category                |
| DELETE | /categories/:id        | Auth   | Delete category                |
| GET    | /transactions          | Auth   | List (filter+paginate+summary) |
| POST   | /transactions          | Auth   | Create transaction             |
| GET    | /transactions/:id      | Auth   | Get transaction                |
| PATCH  | /transactions/:id      | Auth   | Update transaction             |
| DELETE | /transactions/:id      | Auth   | Delete transaction             |
| GET    | /analytics/summary     | Auth   | Dashboard totals               |
| GET    | /analytics/by-category | Auth   | Category breakdown chart       |
| GET    | /analytics/over-time   | Auth   | Trend chart                    |
| GET    | /budgets               | Auth   | List budgets <span>opt</span>  |
| POST   | /budgets               | Auth   | Create budget <span>opt</span> |
| PATCH  | /budgets/:id           | Auth   | Update budget <span>opt</span> |
| DELETE | /budgets/:id           | Auth   | Delete budget <span>opt</span> |

End of API Contract v1.0. Next document: Frontend Technical Requirements (stack, architecture, conventions) — which also becomes the basis for the automated code-review rules.